Software Requirements Specification (SRS)

Cyber Threat Intelligence RAG Chatbot

SRS - CTI RAG Chatbot

Software Requirements Specification (SRS)
Cyber Threat Intelligence RAG Chatbot

Project: CTI RAG Chatbot

Version: 1.0

Date: April 24, 2026

Status: Final

Document Type: Software Requirements Specification

Table of Contents

1. Introduction

1.1 Purpose
This document specifies the functional and non-functional requirements for the Cyber Threat Intelligence RAG Chatbot, a system designed to democratize access to threat intelligence by providing security analysts with conversational, context-grounded answers about MITRE ATT&CK techniques, tactics, and adversary profiles.

1.2 Document Scope
This SRS covers:

Functional requirements for the chatbot system
Non-functional requirements (performance, security, scalability)
System architecture and components
Data requirements and MITRE ATT&CK integration
User interface and API specifications
System constraints and limitations
Quality attributes and acceptance criteria

1.3 Intended Audience
This document is intended for:

Development team members
Project stakeholders and management
Quality assurance and testing team
System architects and technical leads
End users and security operations center (SOC) personnel

1.4 Project Overview
The CTI RAG Chatbot is a retrieval-augmented generation (RAG) system that leverages:

MITRE ATT&CK Framework: Authoritative cybersecurity knowledge base with 200+ techniques, 100+ threat groups, and tactical mitigations
Vector Embeddings: Semantic understanding of threat intelligence using the all-MiniLM-L6-v2 embedding model (384-dimensional vectors)
FAISS Retrieval: Fast approximate nearest neighbor search for document similarity
Large Language Models: Groq cloud LLM for grounded, conversational responses
Hallucination Guards: Validation mechanisms to prevent false threat intelligence

2. Overall Description

2.1 Product Perspective

The CTI RAG Chatbot operates as a standalone web service accessible through a REST API and static web interface. It integrates with:

MITRE ATT&CK Data: STIX JSON format, downloaded and indexed locally
Groq API: Cloud LLM service for natural language generation
HuggingFace Models: Local embedding model for semantic search
FAISS Library: Open-source vector database for indexing

2.2 Product Functions

Function
Description
Priority

Conversational Query Interface
Accept natural language questions about MITRE ATT&CK techniques, tactics, adversaries, and mitigations
Critical

Semantic Retrieval
Find most relevant MITRE ATT&CK documents using embedding-based similarity search
Critical

Grounded Response Generation
Generate answers using LLM augmented with retrieved context
Critical

Source Attribution
Return citations (URLs, technique IDs) for all responses
High

Hallucination Prevention
Validate responses are grounded in retrieved context
High

Chat History Management
Maintain multi-turn conversation context
Medium

Query Rewriting
Redirect offensive queries toward defensive research
Medium

2.3 User Classes and Characteristics

User Class
Characteristics
Usage Pattern

Security Analyst
Conducts threat investigations, medium technical expertise
Tactical lookups, 5-10 queries/day

CTI Specialist
Expert in threat intelligence, deep ATT&CK knowledge
Research synthesis, 20+ queries/day

SOC Manager
Oversees security operations, less technical
Overview queries, ad-hoc usage

Incident Responder
Active incident handling, time-sensitive decisions

Real-time lookups, 50+ queries/day during incidents

2.4 Operating Environment

Deployment: On-premises or cloud virtual machines
OS: Windows, Linux, macOS (Python 3.8+)
Network: Internet required for Groq API (current implementation has no local LLM fallback)
Hardware: Minimum 2GB RAM, 500MB disk (FAISS index)
Browser: Modern browsers supporting HTML5, ES6 JavaScript (Chrome, Firefox, Edge, Safari)

3. Functional Requirements

3.1 Chat API (POST /api/chat)

FR1.1: Accept JSON request with user question

Input: {"question": "What is T1059?"}
Validate that question is non-empty
Force cybersecurity context before retrieval and generation

FR1.2: Perform semantic retrieval

Embed user question using all-MiniLM-L6-v2 model
Query FAISS index for top-4 most similar documents during chat requests
Return retrieved context with similarity scores

FR1.3: Generate LLM response

Invoke Groq API with system prompt + context + question
Support configurable LLM model via GROQ_MODEL env var
Enforce max 4096 tokens output
Set temperature to 0.3 for focused, consistent answers

FR1.4: Validate response groundedness

Apply relevance gating based on retrieval score thresholds
If relevance is low: clear retrieval context and still generate a cybersecurity-scoped response
Inject exact ATT&CK context for explicit T-ID/APT queries when available

FR1.5: Return structured response

Server-Sent Events stream: initial metadata events plus incremental answer chunks
Include MITRE source attribution: name, type, URL, snippet
Include contextual OWASP guidance references (title, URL, reason)
Terminate stream with [DONE] marker

3.2 Reset Endpoint (POST /api/reset)

FR2.1: Clear chat history

Clear session's chat_history array
Return success confirmation
Maintain session isolation across users

3.3 Web Interface

FR3.1: Chat UI rendering

Display conversational messages (user/bot bubbles)
Show source citations with URLs below responses
Display loading indicators during API calls
Support markdown rendering in responses

FR3.2: Query suggestions

Show example queries as clickable chips
Auto-populate input field on chip click

FR3.3: Error handling

Display API errors to user in non-technical language
Retry button for failed requests
Connection status indicator

3.4 Data Ingestion Pipeline

FR4.1: MITRE ATT&CK data loading

Load STIX JSON from local file if available
Otherwise, download from official MITRE GitHub repository
Cache locally for future runs
Support manual STIX file upload

FR4.2: STIX object filtering

Filter to relevant types: attack-pattern, course-of-action, intrusion-set, malware, tool
Extract metadata: ID, name, type, description, URL

FR4.3: Text chunking

Split documents into 1000-character chunks
Apply 200-character overlap for context preservation
Preserve semantic boundaries

FR4.4: Embedding generation and indexing

Generate embeddings using all-MiniLM-L6-v2 model (384-dim)
Create FAISS index with approximate nearest neighbor search
Save index to disk for persistence
Support index versioning and updates

4. Non-Functional Requirements

4.1 Performance Requirements

Requirement
Target
Measurement

Query Latency (Groq)

Time from API call to response

FAISS Retrieval

Vector search time

Embedding Generation

Time to embed query

Throughput
>= 10 req/sec
Concurrent user queries

Index Build Time

First-run ingestion

4.2 Scalability Requirements

Data Scale: Support 1200+ MITRE ATT&CK objects, 3500+ chunks
Concurrent Users: Support at least 10 concurrent API requests
Index Size: FAISS index
Memory: Peak memory usage

4.3 Availability & Reliability

Uptime: Target 99% availability during business hours
Error Recovery: Fail fast at startup with clear error if Groq API is unavailable
Data Persistence: FAISS index persisted to disk, survives application restart
Session Management: Session data retained for conversation context

4.4 Security Requirements

Requirement
Specification

Input Validation
Reject empty question payloads; enforce cybersecurity-scoped prompt construction

API Key Protection
Store GROQ_API_KEY in environment variables, never in source code

HTTPS/TLS
Enforce HTTPS in production; support certificate pinning

Rate Limiting
Not implemented in current codebase; recommended for production reverse proxy or gateway

Data Privacy
LLM prompts are sent to Groq cloud; embeddings and FAISS retrieval remain local

Query Rewriting
Force cybersecurity context and expand short ATT&CK entity queries for retrieval

4.5 Usability Requirements

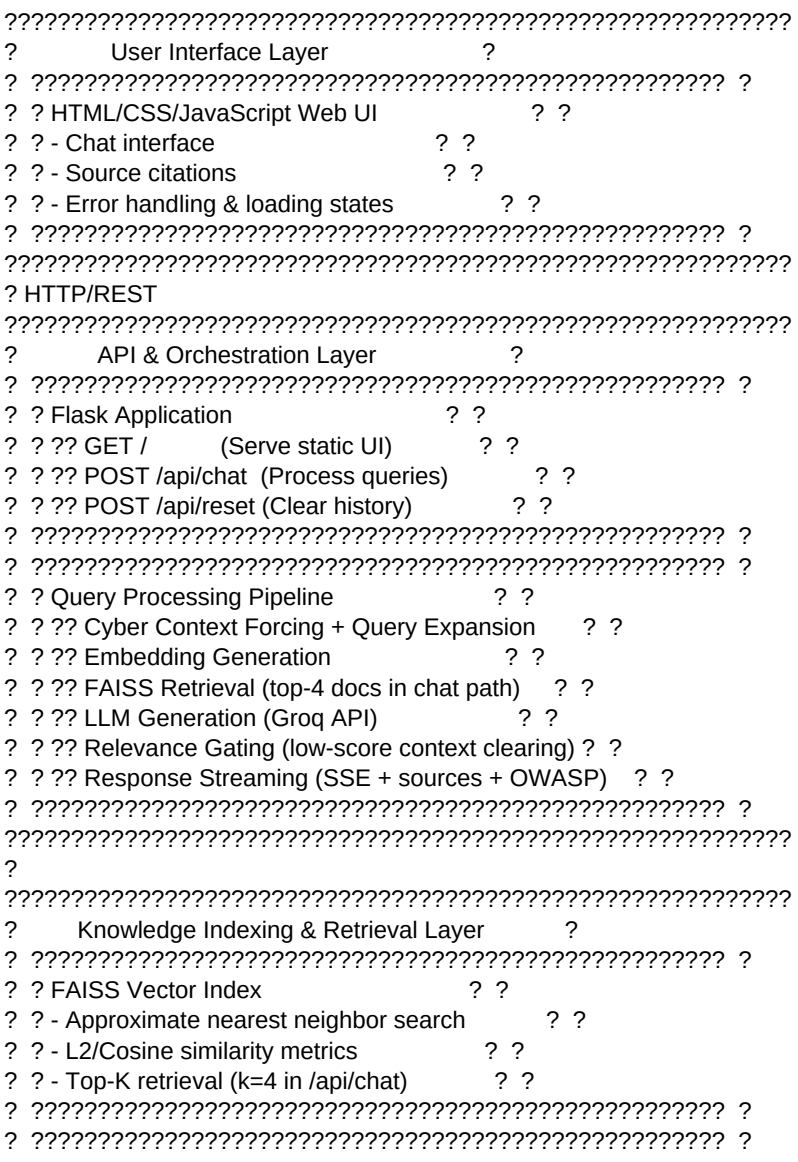
Responsive Design: Web UI works on desktop, tablet, mobile
Accessibility: WCAG 2.1 AA compliance (alt text, keyboard navigation)
Error Messages: User-friendly, non-technical language
Documentation: README, API docs, deployment guide included

4.6 Maintainability Requirements

Code Quality: Follow PEP 8 Python style guide
Testing: Unit tests for core functions (>= 80% coverage)
Logging: Structured logging for debugging and monitoring
Configuration: Externalize settings via environment variables
Documentation: Inline code comments, docstrings for functions

5. System Architecture

5.1 High-Level Architecture



? ? Embedding Model ? ?
? ? - all-MiniLM-L6-v2 (384-dim) ? ?
? ? - Local execution (HuggingFace) ? ?
? ??? ?
? ??? ?
? ? Document Store ? ?
? ? - Chunked MITRE ATT&CK documents ? ?
? ? - Metadata (ID, name, type, URL) ? ?
? ??? ?
??
?
??
? Data Source & External APIs ?
? ?? MITRE ATT&CK STIX JSON ?
? ?? Groq LLM API (cloud inference) ?
? ?? HuggingFace Model Hub (embeddings) ?
??

5.2 Component Descriptions

5.2.1 Flask Backend (backend.py)

Purpose: REST API server and request handler
Endpoints: GET /, POST /api/chat, POST /api/reset, GET /api/status
Responsibilities:

Load FAISS index and embedding model on startup
Handle incoming JSON requests
Orchestrate RAG pipeline
Manage chat history per session
Return structured responses

Dependencies: Flask, LangChain, FAISS, HuggingFace, Groq

5.2.2 Data Ingestion (ingest.py)

Purpose: Prepare MITRE ATT&CK data for retrieval
Process:

Download or load local STIX JSON
Filter to relevant STIX object types
Extract metadata and descriptions
Chunk text with overlap
Generate embeddings
Build FAISS index

Output: faiss_index/index.faiss (persisted on disk)
Frequency: Run once before first server startup, or on data updates

5.2.3 Web UI (static/index.html)

Purpose: User-facing chatbot interface
Features:

Chat message display (user/bot bubbles)
Input field with query suggestions
Source citations with links
OWASP guidance cards and sidebar reference links
Loading indicators and error messages
Responsive design

Technologies: HTML5, CSS3, Vanilla JavaScript (no frameworks)

5.2.4 FAISS Index

Purpose: Fast semantic similarity search
Content: Embeddings of 3500+ document chunks
Size: ~50MB on disk
Search Metric: L2 (Euclidean) distance
Query Time:

5.2.5 External Services

Groq API: Cloud LLM service (llama-3.1-8b-instant default, configurable via env var)
HuggingFace Hub: Pre-trained embedding models
MITRE GitHub: STIX data source

6. Data Requirements

6.1 Input Data

Data Source
Format
Content
Update Frequency

MITRE ATT&CK
STIX 2.1 JSON
200+ techniques, 100+ groups, 600+ malware, 300+ mitigations
Quarterly

User Queries
Plain text (non-empty query required)
Natural language questions
Real-time

6.2 Processing Data

Embedded Chunks: 3500+ document chunks, each 384-dimensional vector
Chat History: Per-session array of {role, content} messages
Metadata: MITRE object ID, name, type, URL for each document

6.3 Output Data

Output
Format
Content

Chat Response
JSON
{success, answer, sources[], context, model}

Retrieved Sources
JSON Array
[{id, name, type, url}]

Error Response
JSON
{success: false, error_message}

6.4 Data Storage

FAISS Index: faiss_index/index.faiss (binary format, ~50MB)
Metadata: In-memory during runtime
Chat History: Session-based, in-memory (cleared on reset or session end)
STIX Cache: data/enterprise-attack.json (local copy for offline access)

7. Interface Requirements

7.1 REST API Specification

7.1.1 POST /api/chat

Request:
Headers: Content-Type: application/json
Body: {
"question": "What is T1059?"
}

Response (Success - SSE stream):
Status: 200 OK
Body events (data: ...):

{"sources": [{"name": "...", "type": "...", "url": "...", "snippet": "..."}]}
{"owasp_refs": [{"title": "...", "url": "...", "reason": "..."}]}
{"chunk": "partial text"}
...
[DONE]

Response (Error):
Status: 500 Internal Server Error
Body: {
"error": "FAISS index not found. Run ingest.py first."
}

7.1.2 POST /api/reset

Request:
Headers: Content-Type: application/json
Body: {}

Response:
Status: 200 OK
Body: {
"status": "reset"
}

7.1.3 GET /

Request:
No body

Response:
Status: 200 OK
Content-Type: text/html
Body: [static/index.html content]

7.2 User Interface Requirements

Layout: Chat interface with message display, input field, and suggestion chips
Message Display: Left-aligned user messages, right-aligned bot responses
Source Citations: Clickable links to MITRE ATT&CK URLs
Loading State: Spinner or progress indicator during API call
Error Handling: Non-technical error messages
Responsive: Works on desktop (1024+px), tablet (768px), mobile (320px)

8. Constraints & Limitations

8.1 Technical Constraints

Constraint
Implication

FAISS In-Memory Index
Cannot scale beyond available RAM; limited to ~1B vectors

LLM Context Window (32K tokens)
Cannot process entire MITRE ATT&CK corpus simultaneously

English-Only Data
MITRE ATT&CK is English; system not multilingual

Groq API Dependency
Offline operation unavailable; cloud outages block service

Local Embeddings
all-MiniLM-L6-v2 is smaller; may miss subtle semantic nuances

8.2 Data Limitations

Knowledge Cutoff: MITRE ATT&CK data refreshed quarterly; zero-day techniques lag
Non-Technical Data: Cannot ingest binary files, network captures, or images
Scope: Limited to MITRE ATT&CK; excludes CVEs, threat reports, indicators

8.3 Operational Constraints

Hardware: Minimum 2GB RAM for comfortable operation
Network: Internet required for Groq API; no offline LLM fallback in production
Scalability: Single-server deployment; horizontal scaling requires external load balancer

8.4 Known Limitations

No multi-modal data support (text-only)
No knowledge graph reasoning (flat retrieval only)
Hardcoded query rewriting (not learned from user feedback)
No user authentication (assumes trusted internal network)
No audit logging of queries

9. Quality Attributes

9.1 Reliability

Mean Time Between Failures (MTBF): > 720 hours (30 days)
Mean Time to Recovery (MTTR):
Failure Modes:

Groq API timeout -> graceful error message
Missing FAISS index -> startup error with recovery instructions
Network disconnection -> connection retry logic

9.2 Testability

Unit Tests: >= 80% code coverage
Integration Tests: End-to-end chat flow validation
Manual UAT: Query variations, error scenarios, edge cases
Performance Tests: Latency benchmarking under load

9.3 Maintainability

Code Quality: PEP 8 compliance, SonarQube score >= 80
Documentation: README, API docs, deployment guide, architecture diagrams
Logging: Structured logs (DEBUG, INFO, WARN, ERROR levels)
Configuration: Environment variables for all settings

9.4 Correctness (Threat Intelligence Accuracy)

Hallucination Rate:
Source Accuracy: 100% of sources must exist in MITRE ATT&CK
Fact-Checking: Manual validation on critical CTI queries
Attribution: All claims must cite source documents

10. Acceptance Criteria

10.1 Functional Acceptance Criteria

Criterion
Success Condition

Chat Queries
User submits question, receives grounded answer with citations

Multi-turn Conversation
System maintains context across 5+ consecutive messages

Source Attribution
All responses include MITRE ATT&CK URLs and technique IDs

Error Handling
Missing FAISS index shows recovery instructions; API errors display gracefully

Query Rewriting
Offensive query "How to hack X?" redirects to defensive context

MITRE Data Ingestion
ingest.py successfully indexes 1200+ STIX objects in

10.2 Performance Acceptance Criteria

Metric
Threshold
Test Method

Query Latency (Groq)

10 representative queries, measure mean latency

Concurrent Users
>= 5 simultaneous requests
Load test with Apache JMeter

Memory Usage

Monitor during 1-hour test

10.3 Security Acceptance Criteria

No hardcoded API keys in source code
GROQ_API_KEY loaded from environment variable
Empty questions are rejected with 400 responses
HTTPS enforced in production
Rate limiting configured at deployment edge (recommended)

10.4 Documentation Acceptance Criteria

README includes setup, usage, testing instructions
API documentation covers all endpoints with examples
Architecture diagram clearly shows component interactions
Deployment guide for production environments

10.5 UAT Sign-off Criteria

Security Analyst: Can query MITRE techniques and receive accurate answers
CTI Specialist: Multi-turn queries work with proper context synthesis
SOC Manager: System provides trustworthy threat intelligence with citations
Developers: Code is maintainable, well-documented, with 80%+ test coverage

Software Requirements Specification v1.0

CTI RAG Chatbot Project

Generated: April 24, 2026

Status: Final